Experiment 18: Docker Image Push to Docker Hub

Objective: To understand Docker registries and learn how to push a Docker image to Docker Hub for sharing and deployment.

Step1. Create Dockerfile and Java File and save both in a folder by the name Docker-Java

Dockerfile

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY . /app
RUN javac Grade.java
CMD ["java", "Grade"]
```

Java File

```
import java.util.Scanner;

public class Grade{

  // Returns a letter grade based on the average
  static char gradeFunction(double avg) {
    if (avg >= 90) return 'A';
    else if (avg >= 80) return 'B';
    else if (avg >= 70) return 'C';
    else if (avg >= 60) return 'D';
    else return 'F';
  }

  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.print("How many grades (1 to 5)? ");
    int count = scanner.nextInt();

    // Validate the input count
    if (count < 1 || count > 5) {
      System.out.println("Invalid number. You must enter between 1 and 5 grades.");
      scanner.close();
      return; // Exit
    }

    double sum = 0.0;
```

```
    // Read each grade
    for (int i = 1; i <= count; i++) {
      System.out.print("Enter grade " + i + ": ");
      double grade = scanner.nextDouble();
      sum += grade;
    }

    double avg = sum / count;
    System.out.println("Average: " + avg);
    System.out.println("Letter grade: " + gradeFunction(avg));

    scanner.close();
  }
}
```

Build the Docker image:

docker build -t java-app .

Step 2. Login to Docker Hub

docker login

Enter your Docker Hub username and password

Step 3: Tag the Docker Image

Tag the image with your Docker Hub repository:

docker tag java-app yourusername/java-app:v1

Step 4: Push Image to Docker Hub

docker push yourusername/java-app:v1

Step 5: Verify on Docker Hub

- Go to your Docker Hub account
- Check repository → Image should be uploaded

Experiment 19: Jenkins and Docker Integration

Objective: To implement Continuous Integration (CI) by integrating **Jenkins** with Docker for automated image building.

Step 1: Create a Jenkins Pipeline Job

- Go to Jenkins Dashboard
- Click **New Item → Pipeline → OK**

Step 2: Add Pipeline Script

```
pipeline {
    agent any
    stages {
        stage('Clone Code') {
            steps {
                git 'https://github.com/your-repo/sample-app.git'
            }
        }
        stage('Build Docker Image') {
            steps {
                bat 'docker build -t my-app .'
            }
        }
        stage('Run Container') {
            steps {
                bat 'docker run -d my-app'
            }
```

```
            }

        }

    }
```

Step 5: Build the Pipeline

- Click **Build Now**
- Monitor in **Console Output**

Step 6: Verify Execution

Check Docker images:

docker images

Check running containers:

docker ps

OR

1. Create the job in jenkins by the name Docker Integration with jenkins

2. Configure the settings like the other jenkins programs

3. Create Repository in github and place the Dockerfile and Java File

4. Write the following Commands in Execute Windows batch command Option in jenkins

docker build -t java-app .
docker run -d -t java-app

5. Save and apply

6. Once build is success check the image created in docker desktop

Experiment 20: Simple End-to-End CI/CD Pipeline

Objective: To understand the DevOps lifecycle by building a simple end-to-end CI/CD pipeline using Jenkins and Docker.

Prerequisites

- Git repository with application code
- Jenkins installed and configured
- Docker installed
- Docker Hub account (optional for deployment)

Tools Used

- Jenkins
- Docker
- Git
- Docker Hub

DevOps Lifecycle Stages Covered

1. **Code** → Developer pushes code to Git
2. **Build** → Jenkins builds the project
3. **Test** → (Optional) Run tests
4. **Package** → Create Docker image
5. **Deploy** → Run container
6. **Monitor** → Check logs/output

Procedure / Tasks

Step 1: Create a Git Repository

- Add application code
- Include a **Dockerfile**
- Include a Jenkinsfile
- Include pom.xml

```
jenkins-tomcat-maven-tested
│
├── src/main/java/com/bnmit/HelloServlet.java
├── src/main/webapp/index.jsp
```

```
├── src/main/webapp/WEB-INF/web.xml
├── pom.xml
├── Jenkinsfile
└── README.txt
```

- **Java Servlets code** (**HelloServlet.java**)

```
package com.example;
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h2>Deployment Successful!</h2>");
        out.println("<h3>GitHub: &rarr; Jenkins: &rarr; Docker: &rarr; Tomcat</h3>");
    }
}
```

<u>Web.xml</u>

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
version="3.1">
<servlet>
<servlet-name>HelloServlet</servlet-name>
```

```
<servlet-class>com.example.HelloServlet</servlet-class>

</servlet>

<servlet-mapping>

<servlet-name>HelloServlet</servlet-name>

<url-pattern>/hello</url-pattern>

</servlet-mapping>

</web-app>
```

Index.jsp

```
<html>

<head>

<title>CI/CD Demo</title>

</head>

<body>

<h1>Welcome to CI/CD Pipeline Demo</h1>

<a href="hello">Click Here</a>

</body>

</html>
```

HelloServletTest.java

```
package com.example;

import static org.mockito.Mockito.*;

import java.io.PrintWriter;

import java.io.StringWriter;
```

```
import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

import org.junit.jupiter.api.Test;

public class HelloServletTest {

    @Test

    void testDoGet() throws Exception {

        // Create servlet instance

        HelloServlet servlet = new HelloServlet();

        // Mock request and response

        HttpServletRequest request = mock(HttpServletRequest.class);

        HttpServletResponse response = mock(HttpServletResponse.class);

        // Capture output

        StringWriter stringWriter = new StringWriter();

        PrintWriter writer = new PrintWriter(stringWriter);

        when(response.getWriter()).thenReturn(writer);

        // Call servlet method

        servlet.doGet(request, response);

        writer.flush();

        String result = stringWriter.toString();

        // Assertions

        assert(result.contains("Deployment Successful!"));

        assert(result.contains("GitHub"));

        assert(result.contains("Jenkins"));

        assert(result.contains("Docker"));
```

```
        assert(result.contains("Tomcat"));

        // Verify response type set

        verify(response).setContentType("text/html");

    }

}
```

Step 2: Create Freestyle Job

- Open Jenkins Dashboard
- Click **New Item → Freestyle Project**

Step 3: Configure Pipeline Script (Jenkins File) in Github

✅ Complete CI/CD Pipeline

```
pipeline {

  agent any

  tools {

    maven 'mymaven'

    jdk 'myjdk'

  }

  environment {

    IMAGE_NAME = "sample-webapp"

    CONTAINER_NAME = "sample-webapp-container"

  }

  stages {

    stage('Clone') {

      steps {

        echo 'Cloning repository'

      }

    }
```

```
        stage('Build WAR') {
            steps {
                bat 'mvn clean package'
            }
        }
        stage('Build Docker Image') {
            steps {
                bat 'docker build -t  sample-webapp .'
            }
        }
        stage('Stop Old Container') {
            steps {
                bat 'docker stop sample-webapp-container || exit 0'
                bat 'docker rm sample-webapp-container || exit 0'
            }
        }
        stage('Run Container') {
            steps {
                bat 'docker run -d -p 8087:8087 --name  sample-webapp-container sample-webapp '
            }
        }


    }
}
```

Step 4: Run the Pipeline

- Click **Build Now**
- Monitor in **Console Output**

Step 5: Verify Deployment

Check running containers:

```
docker ps
```

Access application using two ways:

1.download the generated war file from jenkins and deploy it to tomcat server

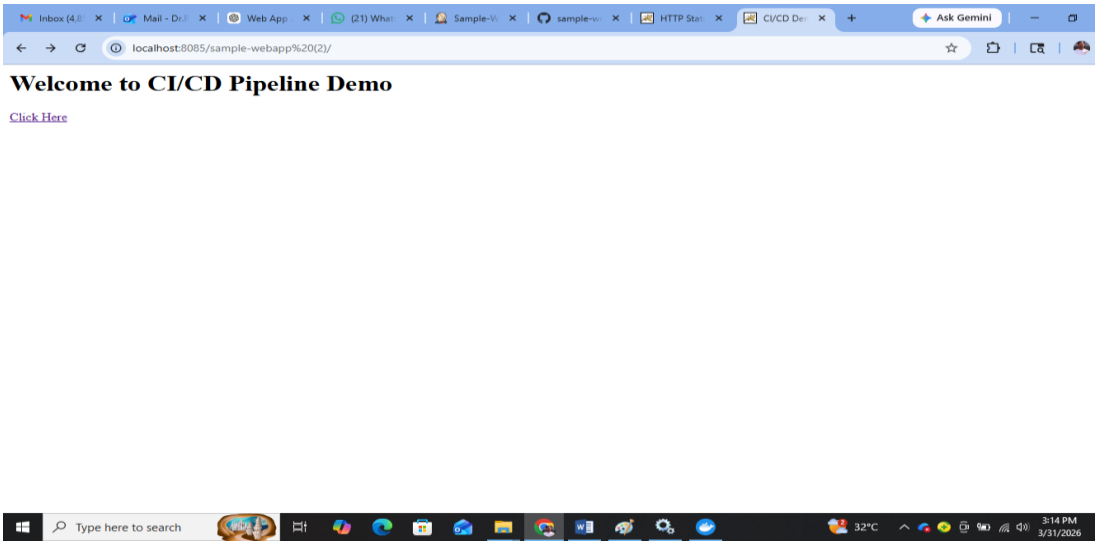
Install tomcat

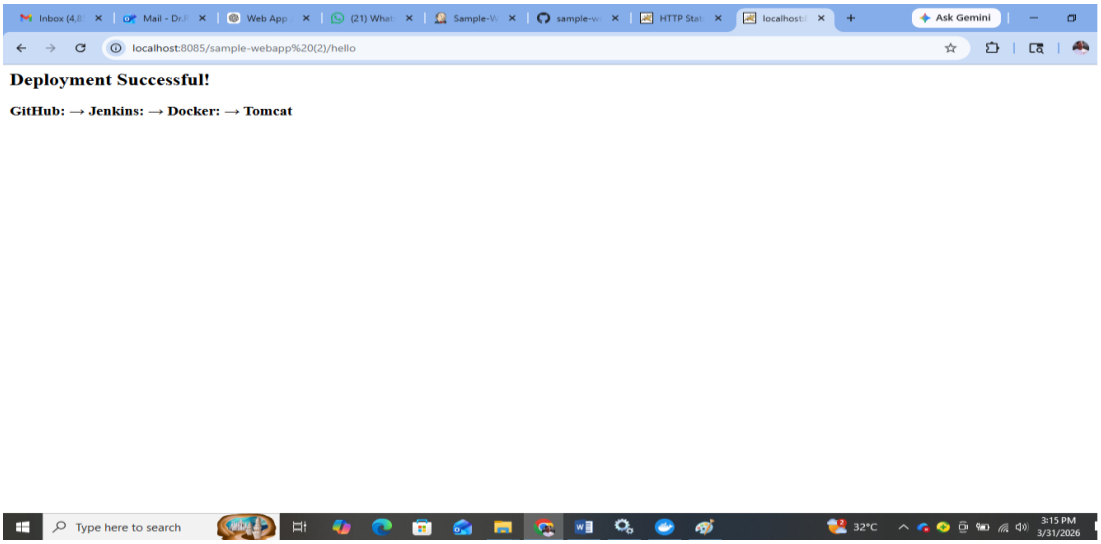
Then access it using localhost

http://localhost:8087

go to Manager App in tomcat

Click on WAR file to deploy Choose file and select the war file downloaded
Click on Deploy





2. Access using Localhost
http://localhost:8087